In [ ]:
```
import marimo as mo
```

Lógica Computacional — Multiplicação de Inteiros

Resolução organizada segundo a ficha **Verificação Indutiva**. O modelo usa `BitVec` de 8 bits e o solver Z3.

Estados do programa: `INIT`, `TEST`, `END` e `ERROR`.

In [ ]:
```
from z3 import (
    BitVec, BitVecVal, Int, Solver, ULT, ULE, UGT,
    ZeroExt, Extract, LShR, And, Or, Not, sat, unsat
)

n = 8
INIT, TEST, END, ERROR = range(4)
```

Alínea a — Construção do SFOTS

Cada estado contém o contador de programa (`pc`), os valores de entrada (`a`, `b`) e as variáveis de trabalho (`x`, `y`, `z`).

In [ ]:
```
def declare(i):
    return {
        "pc": Int(f"pc_{i}"),
        "a": BitVec(f"a_{i}", n),
        "b": BitVec(f"b_{i}", n),
        "x": BitVec(f"x_{i}", n),
        "y": BitVec(f"y_{i}", n),
        "z": BitVec(f"z_{i}", n),
    }
```

In [ ]:
```
def init(state):
    return And(
        state["pc"] == INIT,
        state["x"] == state["a"],
        state["y"] == state["b"],
        state["z"] == BitVecVal(0, n),
    )
```

In [ ]:
```
def trans(curr, prox):
    # Os valores de entrada não se alteram durante a execução.
    inputs = And(
        prox["a"] == curr["a"],
        prox["b"] == curr["b"],
    )
```

```
    # O shift à esquerda causa overflow se o bit mais significativo for 1.
    overflow_shift = (
        Extract(n - 1, n - 1, curr["x"]) == BitVecVal(1, 1)
    )

    # A soma é alargada para n+1 bits para detetar overflow.
    overflow_add = UGT(
        ZeroExt(1, curr["z"]) + ZeroExt(1, curr["x"]),
        BitVecVal(2**n - 1, n + 1),
    )

    return Or(
        # INIT -> TEST
        And(
            curr["pc"] == INIT,
            prox["pc"] == TEST,
            inputs,
            prox["x"] == curr["x"],
            prox["y"] == curr["y"],
            prox["z"] == curr["z"],
        ),

        # TEST -> END quando y = 0
        And(
            curr["pc"] == TEST,
            curr["y"] == BitVecVal(0, n),
            prox["pc"] == END,
            inputs,
            prox["x"] == curr["x"],
            prox["y"] == curr["y"],
            prox["z"] == curr["z"],
        ),

        # TEST -> ERROR: overflow no shift
        And(
            curr["pc"] == TEST,
            curr["y"] != BitVecVal(0, n),
            (curr["y"] & BitVecVal(1, n)) == BitVecVal(0, n),
            overflow_shift,
            prox["pc"] == ERROR,
            inputs,
            prox["x"] == curr["x"],
            prox["y"] == curr["y"],
            prox["z"] == curr["z"],
        ),

        # TEST -> TEST: y é par e o shift é seguro
        And(
            curr["pc"] == TEST,
            curr["y"] != BitVecVal(0, n),
            (curr["y"] & BitVecVal(1, n)) == BitVecVal(0, n),
            Not(overflow_shift),
            prox["pc"] == TEST,
            inputs,
            prox["x"] == curr["x"] << 1,
```

```
            prox["y"] == LShR(curr["y"], 1),
            prox["z"] == curr["z"],
        ),

        # TEST -> ERROR: overflow na soma
        And(
            curr["pc"] == TEST,
            (curr["y"] & BitVecVal(1, n)) == BitVecVal(1, n),
            overflow_add,
            prox["pc"] == ERROR,
            inputs,
            prox["x"] == curr["x"],
            prox["y"] == curr["y"],
            prox["z"] == curr["z"],
        ),

        # TEST -> TEST: y é ímpar e a soma é segura
        And(
            curr["pc"] == TEST,
            (curr["y"] & BitVecVal(1, n)) == BitVecVal(1, n),
            Not(overflow_add),
            prox["pc"] == TEST,
            inputs,
            prox["x"] == curr["x"],
            prox["y"] == curr["y"] - BitVecVal(1, n),
            prox["z"] == curr["z"] + curr["x"],
        ),

        # END é um estado terminal (permanece em END).
        And(
            curr["pc"] == END,
            prox["pc"] == END,
            prox["a"] == curr["a"],
            prox["b"] == curr["b"],
            prox["x"] == curr["x"],
            prox["y"] == curr["y"],
            prox["z"] == curr["z"],
        ),

        # ERROR é um estado terminal (permanece em ERROR).
        And(
            curr["pc"] == ERROR,
            prox["pc"] == ERROR,
            prox["a"] == curr["a"],
            prox["b"] == curr["b"],
            prox["x"] == curr["x"],
            prox["y"] == curr["y"],
            prox["z"] == curr["z"],
        ),
    )
```

In [ ]:

```
# Teste simples: verificar se existe pelo menos uma transição a partir
# de um estado inicial.
estado0 = declare(0)
estado1 = declare(1)
```

```
solver_teste = Solver()
solver_teste.add(init(estado0))
solver_teste.add(trans(estado0, estado1))

resultado_teste = solver_teste.check()
print("Existe uma transição possível a partir de um estado inicial?",
      resultado_teste)

if resultado_teste == sat:
    print("Modelo:", solver_teste.model())
```

```
Existe uma transição possível a partir de um estado inicial? sat
Modelo: [a_1 = 0,
 x_1 = 0,
 b_1 = 0,
 b_0 = 0,
 z_0 = 0,
 y_1 = 0,
 y_0 = 0,
 pc_0 = 0,
 x_0 = 0,
 pc_1 = 1,
 a_0 = 0,
 z_1 = 0]
```

Alínea b — Verificação do invariante por indução

Pretende-se verificar:

$$\phi(s) \equiv x \times y + z = a \times b$$

Seguindo a ficha, verificamos duas condições:

- **Caso base:** não existe estado inicial que viole o invariante.
- **Passo indutivo:** não existe transição de um estado que satisfaz o invariante para outro estado que o viole.

In [ ]:
```
def induction_always(declare, init, trans, inv):

    state = declare(0)
    state_next = declare(1)

    # Caso base
    solver_base = Solver()
    solver_base.add(
        And(init(state), Not(inv(state)))
    )

    resultado_base = solver_base.check()

    if resultado_base == sat:
        print("Falha no caso base")
        print(solver_base.model())
        return False
```

```
    # Passo indutivo
    solver_passo = Solver()
    solver_passo.add(
        And(
            inv(state),
            trans(state, state_next),
            Not(inv(state_next))
        )
    )

    resultado_passo = solver_passo.check()

    if resultado_passo == sat:
        print("Falha no passo indutivo")
        print(solver_passo.model())
        return False

    print("Propriedade verificada com sucesso")
    return True
```

In [ ]:
```
def inv(state):
    return (
        state["x"] * state["y"] + state["z"]
        == state["a"] * state["b"]
    )

resultado_invariante = induction_always(
    declare, init, trans, inv
)
```

```
Propriedade verificada com sucesso
```

Conclusão da alínea b. Se o caso base e o passo indutivo devolverem `unsat`, não existe contraexemplo para nenhuma das duas condições. Logo, o invariante é preservado em todas as execuções do SFOTS.

Alínea c — Verificação da segurança

Restringimos os estados iniciais a:

$$a < 2^{n/2} \quad \wedge \quad b < 2^{n/2}$$

Para $n = 8$, isto corresponde a $a < 16$ e $b < 16$. Usamos um invariante auxiliar que também limita o valor intermédio $x \times y + z$ e exclui o estado `ERROR`.

In [ ]:
```
def init_seguro(state):
    return And(
        init(state),
        ULT(state["a"], BitVecVal(2 ** (n // 2), n)),
        ULT(state["b"], BitVecVal(2 ** (n // 2), n)),
    )
```

In [ ]:
```python
def inv_seguro(state):
    return And(
        # Limites dos inputs
        ULT(state["a"], BitVecVal(2 ** (n // 2), n)),
        ULT(state["b"], BitVecVal(2 ** (n // 2), n)),

        # Propriedade funcional
        state["x"] * state["y"] + state["z"]
        == state["a"] * state["b"],

        # Limite superior do resultado
        ULE(
            ZeroExt(n, state["x"]) * ZeroExt(n, state["y"])
            + ZeroExt(n, state["z"]),
            BitVecVal((2 ** (n // 2) - 1) ** 2, 2 * n),
        ),

        # Segurança
        state["pc"] != ERROR,
    )
```

In [ ]:
```python
resultado_invariante_seguro = induction_always(
    declare, init_seguro, trans, inv_seguro
)
```

```
Propriedade verificada com sucesso
```

Testes de execução

Os testes seguintes verificam exemplos concretos. São complementares à prova por indução: não a substituem.

In [ ]:
```python
#gerada por IA, link no fim do documento
def bmc_always(declare, init, trans, inv, K):
    for k in range(1, K+1):

        s = Solver()

        states = [declare(i) for i in range(k)]

        constraints = []

        constraints.append(init(states[0]))

        for i in range(k-1):
            constraints.append(trans(states[i], states[i+1]))

        constraints.append(Not(inv(states[-1])))

        s.add(And(constraints))

        if s.check() == sat:
            model = s.model()
```

```
            for i, st in enumerate(states):
                pc_val = model.eval(st['pc'])
                x_val = model.eval(st['x'])
                y_val = model.eval(st['y'])
                z_val = model.eval(st['z'])

                print(
                    f"Estado {i}: pc={pc_val}, "
                    f"x={x_val}, y={y_val}, z={z_val}"
                )

            return False

    print(f"Nenhum contraexemplo encontrado até {K}")
    return True

def init_teste(state, a_val, b_val):
    return And(
        init_seguro(state),
        state["a"] == BitVecVal(a_val, n),
        state["b"] == BitVecVal(b_val, n)
    )

print("Teste 1:")
bmc_always(
    declare,
    lambda s: init_teste(s, 5, 0),
    trans,
    inv_seguro,
    30
)

print("Teste 2:")
bmc_always(
    declare,
    lambda s: init_teste(s, 7, 3),
    trans,
    inv_seguro,
    30
)

print("Teste 3:")
bmc_always(
    declare,
    lambda s: init_teste(s, 15, 15),
    trans,
    inv_seguro,
    30
)
```

```
Teste 1:
Nenhum contraexemplo encontrado até 30
Teste 2:
Nenhum contraexemplo encontrado até 30
Teste 3:
Nenhum contraexemplo encontrado até 30
```

True

Alínea d — Propriedade de animação: terminação

Consideramos que o programa termina quando chega a `END` ou `ERROR`. Assim, a propriedade temporal é:

$$F(pc = END \vee pc = ERROR)$$

Tal como na ficha, procuramos um contraexemplo em forma de *loop*: um traço que nunca atinja um estado terminal e cujo último estado possa transitar para um estado anterior.

In [ ]:
```
def terminal(state):
    return Or(
        state["pc"] == END,
        state["pc"] == ERROR,
    )
```

In [ ]:
```
def bmc_eventually(declare, init, trans, prop, bound):
    for k in range(1, bound + 1):
        solver = Solver()
        states = [declare(300 + i) for i in range(k)]

        constraints = [init(states[0])]

        for i in range(k - 1):
            constraints.append(trans(states[i], states[i + 1]))

        for state in states:
            constraints.append(Not(prop(state)))

        loop = [
            trans(states[-1], states[j])
            for j in range(k)
        ]
        constraints.append(Or(loop))

        solver.add(And(constraints))
        resultado = solver.check()

        if resultado == sat:
            model = solver.model()
            print("Foi encontrado um contraexemplo:")
            for i, state in enumerate(states):
                print(
                    f"Estado {i}: pc={model.eval(state['pc'])}, "
                    f"y={model.eval(state['y'])}"
                )

            for j in range(k):
                if solver.check(
                    trans(states[-1], states[j])
                ) == sat:
```

```
                    print(f"O loop regressa ao estado {j}.")
                    break
            return False

    print(f"Nenhum contraexemplo encontrado até {bound} estados.")
    return True

resultado_animacao = bmc_eventually(
    declare, init, trans, terminal, 19
)
```

```
Nenhum contraexemplo encontrado até 19 estados.
```

Justificação da terminação

Em cada passagem pelo estado `TEST`:

- se $y = 0$, o programa transita para `END`;
- se y é par e não nulo, passa para $y/2$;
- se y é ímpar, passa para $y - 1$;
- se ocorrer overflow, transita para `ERROR`.

Portanto, enquanto não chega a um estado terminal, o valor unsigned de y diminui estritamente. Como y é um `BitVec` de 8 bits, começa entre 0 e 255 e não pode diminuir indefinidamente.

O maior número de iterações do ciclo é limitado: para $y = 255$, a sequência decrescente é $255, 254, 127, 126, 63, 62, 31, 30, 15, 14, 7, 6, 3, 2, 1, 0$. Assim, não pode existir um *loop* não terminal. A pesquisa BMC acima procura precisamente esse tipo de contraexemplo.

LLM

link do chat usado como forma de apoio a reslizacao do trabalho:
https://chatgpt.com/share/6ae9d4c4-e588-83ed-bc32-8599443ccc98